

Production Code Audit Verification & Gap Analysis

Comprehensive Codebase Re-Assessment Against the 41-Issue Security & Trading Integrity Audit

AUDIT TARGET	VERIFICATION DATE	ORIGINAL VERDICT	CURRENT VERDICT
Backend + Admin Web + WebView + APK	October 05, 2026	NOT safe for real money	NOT SAFE YET (P0 Gaps Remain)

32

STILL VULNERABLE (OPEN)

5

FULLY RESOLVED

4

PARTIALLY MITIGATED

41

TOTAL ISSUES TRACKED

Executive Assessment: Progress Achieved vs Critical Blockers Remaining

Significant, high-quality progress has been made in the core trading engine: **Race conditions on balance updates and trade square-offs (CON-001 / Bug #8 & Bug #7) have been rigorously solved** using in-memory mutexes (Set) and MySQL row locks (FOR UPDATE). Client-controlled execution prices (FIN-001), client P&L forging (FIN-002), client lot-size tampering (FIN-005), and fractional quantity precision (DB-001) are now successfully enforced server-side.

However, critical security perimeter vulnerabilities remain 100% active: Four root AI routes run without login (SEC-001); the AI mediator has full LLM-to-SQL execution privileges (SEC-002); user listings leak password hashes and plaintext tokens (SEC-003); anyone can reset passwords and alter limits due to missing role guards (SEC-004, SEC-005); SQL injection exists in user date filters (SEC-006); voice audio downloads allow directory traversal to backend .env (SEC-007); and the KYC uploads folder is served publicly (SEC-016).

Audit Status Matrix by Priority (Original Audit vs Current Codebase)

41 Verified Findings

PRIORITY	TOTAL	RESOLVED (FIXED)	PARTIALLY FIXED	STILL OPEN (VULNERABLE)	CURRENT RISK PROFILE
P0 (CRITICAL)	16	4 Fixed (25%)	1 Partial (6%)	11 Open (69%)	Severe capital theft, unauthorized database execution, credential compromise
P1 (HIGH)	15	0 Fixed (0%)	1 Partial (7%)	14 Open (93%)	Socket spoofing, unauthenticated admin actions, lack of rate limits, IDOR
P2 (MEDIUM)	8	1 Fixed (12%)	2 Partial (25%)	5 Open (63%)	N+1 list queries, unindexed writes, error leakage, public static uploads
P3 (LOW)	2	0 Fixed (0%)	0 Partial (0%)	2 Open (100%)	Monolithic controller file sizes, duplicated lot-size lookup dictionaries
TOTAL	41	5 (12.2%)	4 (9.8%)	32 (78.0%)	Production deployment blocked pending Phase 1 security remediation

Audit Status Matrix by Category

CATEGORY	TOTAL	FIXED	PARTIAL	OPEN	CATEGORY HEALTH VERDICT
Security	21	0	0	21	CRITICAL ATTACK SURFACE — Immediate remediation required

CATEGORY	TOTAL	FIXED	PARTIAL	OPEN	CATEGORY HEALTH VERDICT
Trading / Money Correctness	8	3	2	3	SUBSTANTIALLY IMPROVED — Core pricing safe; ledger/delete gaps remain
Reliability	6	1	0	5	MODERATE STABILITY — Concurrency safe; fragile crons & single session
Database	2	1	1	0	HEALTHY — Decimal quantities implemented; index hygiene in progress
Performance	2	0	0	2	HIGH DB LATENCY — N+1 queries, correlated subqueries, 50MB body limit
Code Quality	2	0	0	2	TECHNICAL DEBT — 3k+ line controllers, hardcoded dictionaries

Detailed Verification Register: P0 Critical Issues (16 Issues)

ID & PRI	AUDIT TITLE	CODE LOCATION	STATUS	CURRENT CODEBASE BEHAVIOR & VERIFICATION EVIDENCE
SEC-001 P0	Four AI routes open with no login	<code>server.js:147-150</code>	STILL OPEN	Routes <code>/ai-parse</code> , <code>/execute-command</code> , <code>/smart-command</code> , <code>/master-command</code> are explicitly bound directly on <code>app.post()</code> at root with comment: <i>"// no /api prefix, no auth required for direct access"</i> . Anyone on the internet can invoke voice/master AI without credentials.
SEC-002 P0	LLM writes & runs raw SQL; no role checks	<code>aiMediator.js:32-95</code> <code>aiMasterExecutor.js</code>	STILL OPEN	<code>aiMediator.js</code> provides OpenAI Function Calling tools: <code>db_read</code> , <code>db_write</code> , <code>db_transaction</code> , accepting arbitrary raw SQL strings and parameters. A prompt injection via chat or voice command can execute unauthorized UPDATE, DELETE, or DROP statements.
SEC-003 P0	User endpoints leak password hashes & tokens	<code>userController.js:44</code> <code>userController:154</code>	STILL OPEN	<code>getUsers</code> runs <code>SELECT u.*</code> which dumps the entire user record, including <code>bcrypt password hashes</code> , <code>transaction_password</code> , and <code>session_token</code> . In addition, <code>getUserProfile</code> (GET <code>/users/:id</code>) performs no ownership or hierarchy check.
SEC-004 P0	Anyone can change/reset anyone's password	<code>userRoutes.js:21</code> <code>userController:250</code>	STILL OPEN	<code>router.put('/:id/passwords')</code> uses <code>brokerPermission('createClientsAllowed')</code> which immediately calls <code>next()</code> for any non-BROKER role (e.g. TRADER). Thus any client trader can change passwords on any account. <code>resetPassword</code> executes without current-password verification.
SEC-005 P0	Any user can edit limits/KYC/brokerage	<code>userRoutes.js:27,28,33</code> <code>userController.js</code>	STILL OPEN	<code>PUT /:id</code> , <code>PUT /:id/settings</code> , and <code>PUT /:id/documents</code> lack <code>roleMiddleware</code> . Client TRADERS bypass the broker check and can update their own credit limit, modify trading segment permissions, or self-verify KYC status.
SEC-006 P0	SQL injection in user-list date filter	<code>userController.js:22-29</code>	STILL OPEN	Unanchored regex <code>/^d{4}-d{2}-d{2}/.test(fromDate)</code> validates only the date prefix. The raw string is directly concatenated: <code>tradeDateFilter += ' AND ... >= ' + fromDate</code> . An input like <code>2026-01-01' OR 1=1 -</code> injects raw SQL syntax directly.
SEC-007 P0	Path traversal in voice-audio download	<code>voiceRecordingController.js:216</code>	STILL OPEN	<code>getAudio</code> executes: <code>path.join(__dirname, '../uploads/recordings', filename)</code> without stripping directory traversals or calling <code>path.basename()</code> . An attacker can request <code>/api/ai/voice/audio/..%2F..%2F.env</code> and download all server credentials.
SEC-008 P0	Any user controls company Zerodha session	<code>kiteRoutes.js:47,50,56</code> <code>kiteController:139</code>	STILL OPEN	<code>/set-token</code> , <code>/auto-login</code> , and <code>/disconnect</code> have no role checks (any TRADER can call them). In <code>kiteController.callback</code> , the generated Zerodha <code>access_token</code> is echoed directly into public browser HTML inside a <code><textarea></code> element.

ID & PRI	AUDIT TITLE	CODE LOCATION	STATUS	CURRENT CODEBASE BEHAVIOR & VERIFICATION EVIDENCE
SEC-009 P0	Secrets & PII committed; TOTP logged	<code>src/data/kite_session.json</code> <code>KiteAutoLoginService:202</code>	STILL OPEN	Live user credentials (name, email, user_id, active tokens) are committed in <code>src/data/kite_session.json</code> . In <code>KiteAutoLoginService.js:202</code> , the live 6-digit TOTP code is explicitly printed to standard console logs: <code>console.log('[KiteAutoLogin] Generated 6-digit TOTP code: ' + totpCode)</code> .
FIN-001 P0	Client chooses the execution price	<code>tradeController.js:604-608</code>	RESOLVED	FIXED: For MARKET orders, client-supplied price is strictly ignored: <code>executionPrice = parseFloat(liveMarketPrice)</code> ; If live market data is unavailable from the tick feed, the order is strictly rejected with HTTP 400.
FIN-002 P0	Client sets PnL & exit price on close	<code>tradeController:2505,2608</code> <code>TradeService:208,370</code>	RESOLVED	FIXED: Ownership check prevents closing other users' trades. <code>tradeController</code> passes null for P&L to <code>TradeService.closeTrade</code> . For TRADER requesters, client-sent exit price is discarded and P&L is strictly computed server-side via formula helpers.
FIN-003 P0	Internal transfer can steal money	<code>portfolioController.js:23-130</code>	PARTIALLY FIXED	PARTIALLY FIXED: Negative amounts, zero amounts, and self-transfers are strictly blocked. Sender and receiver are locked FOR UPDATE in deterministic order. Remaining: No hierarchy check prevents transferring funds across unrelated broker hierarchies.
FIN-004 P0	Broken ledger model creates/destroys money	<code>tradeController.js:2670-2681</code>	STILL OPEN	When an OPEN trade is deleted via <code>deleteTrade</code> , <code>balanceRefund = marginToRefund + pnlToRefund</code> executes UPDATE users SET <code>balance = balance + balanceRefund</code> . Because margin was never debited upon entry, deleting an open trade creates money from nothing with zero ledger row.
FIN-005 P0	Client-controlled lot size / leverage	<code>tradeController.js:270-279, 1395</code>	RESOLVED	FIXED: Security guard deletes <code>lot_size_at_entry</code> , <code>leverage_used</code> , <code>margin_used</code> , and <code>exposure</code> from <code>req.body</code> . Lot size is strictly derived from <code>scrip_data</code> / DB cache, and leverage is derived from <code>clientConfig</code> .
CON-001 P0	Race conditions on balance & close	<code>TradeService.js:90-114</code> <code>tradeController:1505</code>	RESOLVED	FIXED: Verified resolved in Bug #7 & Bug #8 audit. Uses fast in-memory mutex <code>this.closingLocks = new Set()</code> to reject concurrent close requests in 0-9ms. Uses MySQL <code>SELECT ... FOR UPDATE</code> with atomic status update guard (<code>affectedRows === 1</code>).
SEC-018 P0	AI write routes under /api/ai reachable by TRADER	<code>aiRoutes.js:30-36</code>	STILL OPEN	Endpoints <code>/api/ai/smart-command</code> , <code>/api/ai/master-command</code> , and <code>/api/ai/mediate</code> require only <code>authMiddleware</code> without <code>roleMiddleware(['ADMIN', 'SUPERADMIN'])</code> . Any trader can execute administrative state changes via the LLM agent.

Detailed Verification Register: P1 High Priority Issues (15 Issues)

ID & PRI	AUDIT TITLE	CODE LOCATION	STATUS	CURRENT CODEBASE BEHAVIOR & VERIFICATION EVIDENCE
REL-005 P1	Pending/Target-SL monitors can double-execute	PendingOrderService.js:90 TradeService.js:856-909	STILL OPEN	executePendingOrderNetting selects pending trade via <code>SELECT t.* FROM trades t WHERE t.id = ? WITHOUT FOR UPDATE</code> and without an atomic status condition guard (<code>WHERE is_pending = 1</code>). If two ticks trigger concurrently, duplicate executions occur.
SEC-010 P1	Socket.IO: unauthenticated connect; trusted identity	SocketManager.js:26-48	STILL OPEN	In <code>SocketManager.init()</code> , missing or invalid JWT tokens simply assign <code>socket.user = null</code> and call <code>next()</code> , permitting unauthenticated WebSockets. <code>socket.on('join', ({ userId, role }) => ...)</code> blindly trusts client input to join arbitrary user/role rooms.
SEC-011 P1	No rate limiting; login user-enumeration	authController.js:19,25 server.js	STILL OPEN	No <code>express-rate-limit</code> middleware is registered on authentication, order placement, or AI routes. <code>authController.login</code> explicitly returns "User not found" vs "Invalid password", enabling rapid brute-force username enumeration.
SEC-012 P1	Weak token lifecycle	auth.js:4 authController.js:77,81	STILL OPEN	<code>authMiddleware</code> accepts tokens from query strings (<code>req.query?.token</code>), leaking credentials into server access logs. JWTs are valid for a fixed 24 hours without rotation, refresh tokens, or revocation blacklists. <code>session_token</code> is stored as plaintext in DB.
SEC-013 P1	Admin actions with weak/no protection	systemRoutes.js:18 scripTickRoutes.js:22 contractRoutes.js:22-26	STILL OPEN	<code>GET /api/system/debug/refresh-market-data</code> has no auth. <code>POST /api/scrip-ticks/cleanup</code> (spawns 4GB purge worker) has no role check. <code>POST /api/contracts/rollover/*</code> (suggestion & enable next) is completely unauthenticated.
SEC-014 P1	Missing hierarchy checks (IDOR) across endpoints	userController.js:154 fundRoutes.js	STILL OPEN	Endpoints accepting <code>:id</code> (such as <code>GET /users/:id</code> , <code>GET /users/:id/documents</code> , <code>GET /users/:id/weekly-balance</code>) fail to verify that the requesting user owns the resource or has an authorized parent/broker hierarchy relationship.
SEC-015 P1	Transaction password optional where it matters	tradeController.js:2655,2749	STILL OPEN	In <code>deleteTrade</code> and <code>updateTrade</code> , password validation executes: <code>if (req.user.role !== 'TRADER' && req.body && req.body.transactionPassword)</code> . If the admin or attacker simply omits <code>transactionPassword</code> from the JSON body, the check is bypassed entirely.
SEC-016 P1	Public file serving; KYC stored publicly	server.js:105 utils/imagekit.js	STILL OPEN	<code>app.use('/uploads', express.static(path.join(__dirname, '../uploads')))</code> mounts the entire uploads directory publicly. User Aadhaar cards, PAN cards, and bank statements uploaded locally or via fallback can be downloaded directly via predictable URLs.

ID & PRI	AUDIT TITLE	CODE LOCATION	STATUS	CURRENT CODEBASE BEHAVIOR & VERIFICATION EVIDENCE
SEC-017 P1	IP spoofing; broken IP logging	<code>server.js:45, 101</code>	STILL OPEN	<code>app.set('trust proxy', true)</code> trusts client-forged X-Forwarded-For headers without hop limits. In line 101, <code>app.use(logIp)</code> is executed before route authentication, where <code>req.user</code> is undefined, leaving IP audit logs incomplete.
FIN-006 P1	Ledger integrity & absolute balance updates	<code>WeeklySettlementService:383</code> <code>fundController.js</code>	PARTIALLY FIXED	PARTIALLY FIXED: Weekly settlement overwrite was updated to delta-based update: <code>balance = balance + ?</code> . However, manual admin fund adjustments and trade deletions update balances without creating corresponding double-entry ledger records.
REL-001 P1	Fragile schedulers	<code>server.js:216-235</code> <code>KiteAutoLoginService:51</code>	STILL OPEN	Independent cron schedules and polling intervals run inside the main process without distributed locking (e.g. Redis or MySQL <code>GET_LOCK</code>). In a clustered or multi-container environment, weekly settlements and auto-logins run simultaneously across instances.
REL-002 P1	Crash handling & health checks	<code>server.js:8-10, 157-159</code>	STILL OPEN	<code>process.on('uncaughtException')</code> logs errors to console without exiting, leaving node memory and database connection states corrupted. The <code>/health</code> endpoint returns static 200 OK without pinging MySQL or validating WebSocket health.
REL-003 P1	Single global Zerodha session; cross-user fallback	<code>KiteAuthService.js:64-70</code> <code>kiteService.js</code>	STILL OPEN	<code>kiteService</code> holds a single global token in memory. <code>KiteAuthService.getStatus</code> falls back to <code>getLatestSession()</code> , showing another user's active session state if the querying user has not connected Zerodha.
LOG-001 P1	Sensitive logging	<code>KiteAutoLoginService:202</code> <code>authController.js:11</code>	STILL OPEN	Live 2FA TOTP codes are printed to stdout on line 202. Login attempt messages print usernames and password lengths. Order execution routes log user balances and full trade request payloads without redaction.

Detailed Verification Register: P2 Medium & P3 Low Issues (10 Issues)

ID & PRI	AUDIT TITLE	CODE LOCATION	STATUS	CURRENT CODEBASE BEHAVIOR & VERIFICATION EVIDENCE
DB-001 P2	trades.qty INT vs fractional netting	<code>migrate.js:110</code>	RESOLVED	FIXED: In <code>migrate.js</code> , <code>trades.qty</code> is defined as <code>DECIMAL(18,4) NOT NULL DEFAULT 0.0000</code> . Fractional quantities from FIFO netting ratios are stored accurately without silent MySQL integer truncation.
DB-002 P2	Index hygiene	<code>migrate.js:1130-1136</code>	PARTIALLY FIXED	PARTIALLY FIXED: Composite indexes (<code>idx_trades_user_status</code> on <code>user_id</code> , <code>status</code> and <code>idx_users_role_status</code>) have been added. However, redundant single-column indexes on high-write tables still cause unnecessary I/O overhead.
PERF-001 P2	Heavy list queries; no pagination	<code>userController.js:42-63</code> <code>requestController:39-49</code>	STILL OPEN	<code>getUsers</code> executes 6 correlated subqueries for every single row returned. <code>getRequests</code> runs $N+1$ queries in loops to compute withdrawable balances. Neither endpoint supports SQL pagination (<code>LIMIT / OFFSET</code>). Body limit is 50MB.
PERF-002 P2	Polling load	<code>server.js:217-220</code>	STILL OPEN	The server maintains 4 active polling intervals (RMS 10s, Target/SL 5s, Alerts 3s, Pending 3s), querying MySQL repeatedly instead of triggering price checks on incoming WebSocket tick streams.
ERR-001 P2	Error leakage	<code>tradeController.js:1760,2645</code>	STILL OPEN	Multiple error handlers return <code>{ error: err.message }</code> directly to API clients. When MySQL errors occur, internal table schema, constraint names, and query fragments are leaked to the caller.
DASH-001 P2	Silent stale-price valuation	<code>RMSService.js:99-110</code>	STILL OPEN	In <code>RMSService.js</code> , if live market price is missing, trade valuation silently falls back to <code>last_market_price</code> or <code>entry_price</code> . This masks feed failures from risk managers and risks incorrect auto-squareoffs.
FE-001 P2	Frontend token storage & artifacts	<code>Trading_Frontend-main/.env</code>	STILL OPEN	JWT tokens are stored in browser <code>localStorage</code> . The production backend URL (<code>https://trading-backend-production-72ac.up.railway.app</code>) is committed in <code>Trading_Frontend-main/.env</code> . Large build zip files (<code>Trading_Frontend.zip</code> , <code>frosted.zip</code>) remain committed in git.
APK-001 P2	Mobile permissions & old-client contract	<code>trading-updated-apk-main/app.json</code>	STILL OPEN	The mobile APK requests excessive Android permissions (<code>RECORD_AUDIO</code> , <code>READ_EXTERNAL_STORAGE</code> , <code>WRITE_EXTERNAL_STORAGE</code> , <code>READ_MEDIA_IMAGES</code> , <code>READ_MEDIA_VIDEO</code> , <code>READ_MEDIA_AUDIO</code>).
REL-004 P2	No backup/monitoring/deploy config; no tests	<code>tests/unit</code> <code>sharemarket-aws-backend-master</code>	PARTIALLY FIXED	PARTIALLY FIXED: Unit tests have been introduced for settlement, brokerage, and P&L helpers. Remaining: No CI/CD pipelines (<code>.github</code> directory absent), no automated

ID & PRI	AUDIT TITLE	CODE LOCATION	STATUS	CURRENT CODEBASE BEHAVIOR & VERIFICATION EVIDENCE
				database backup scripts, and no containerization config in repo.
CQ-001 P3	Huge controllers; junk files; wrong query	tradeController.js:3262 lines userController.js:840	STILL OPEN	tradeController.js exceeds 3,260 lines; kiteRoutes.js exceeds 2,400 lines. In userController.js:840, recalculateBrokerage treats all fallback trades as MCX using config.mcxBrokerageType. Crash stack dumps (grep.exe.stackdump) exist in repos.
CQ-002 P3	Duplicated lot-size tables	kiteController.js:4-19 symbolHelper.js:144-190	STILL OPEN	Hardcoded MCX_LOT_SIZES and NFO_LOT_SIZES maps are duplicated across kiteController.js, symbolHelper.js, and frontend code rather than referencing the authoritative scrip_data table.

Technical Deep Dive on Resolved Items vs Remaining Critical Gaps

✅ Major Milestones Successfully Verified

- **Trading Price Authority (FIN-001):** Verified that MARKET orders in `tradeController.js:604-608` strictly enforce server-side prices (`executionPrice = parseFloat(liveMarketPrice)`). Client-supplied pricing is stripped, preventing artificial low-price buys and instant profits.
- **P&L Authority & Square-Off Ownership (FIN-002):** Verified that `tradeController.closeTrade` rejects requests if a TRADER attempts to close a trade owned by another user. Furthermore, P&L is strictly computed by server-side financial engines (`commodityLotService`, `calculateMcxPnL`, `calculateEquityPnL`), ignoring client-supplied P&L values.
- **Server-Side Lot Size & Leverage Guard (FIN-005):** Verified that `tradeController.js:270-279` explicitly strips client-submitted `lot_size_at_entry`, `leverage_used`, and `exposure`, calculating margin requirements strictly from authoritative database settings.
- **Concurrency Mutex & Row-Level Locks (CON-001):** Verified that `TradeService.js:90-102` utilizes an in-memory Set lock (`this.closingLocks`) to reject concurrent square-off double-clicks in 0-9ms, combined with `SELECT ... FOR UPDATE` and `affectedRows === 1` guards at the database layer.
- **Fractional Netting Precision (DB-001):** Verified that `trades.qty` is configured as `DECIMAL(18,4)` in `migrate.js`, eliminating silent MySQL integer truncations on partial FIFO closures.

🚨 Urgent Action Items (Must Fix Before Any Real-Money Launch)

- **Disable / Authenticate Root AI Routes (SEC-001, SEC-002, SEC-018):** Remove lines 147-150 in `src/server.js (/ai-parse, /execute-command, /smart-command, /master-command)`. Enforce `roleMiddleware(['SUPERADMIN'])` on all AI endpoints under `/api/ai/*` and eliminate raw SQL execution from LLM mediator tools.
- **Sanitize User Endpoints & Enforce IDOR Checks (SEC-003, SEC-004, SEC-005, SEC-014):** Replace `SELECT u.*` in `userController.getUsers` with explicit column selections (excluding `password`, `transaction_password`, and `session_token`). Add `roleMiddleware(['ADMIN', 'SUPERADMIN'])` to `PUT /:id/passwords` and `PUT /:id/settings` to prevent trader privilege escalation.
- **Fix SQL Injection in Date Filters (SEC-006):** In `userController.js:22-29`, replace string concatenation with parameterized SQL bindings (?) and anchor the date regex: `/^d{4}-d{2}-d{2}$/`.
- **Patch Path Traversal & Protect File Uploads (SEC-007, SEC-016):** In `voiceRecordingController.js:216`, sanitize filenames using `path.basename(filename)`. Remove public static serving of `/uploads` in `server.js:105` and serve KYC documents only via authenticated endpoints with ownership checks.
- **Lock Down Zerodha Admin Endpoints & Remove Committed Secrets (SEC-008, SEC-009):** Restrict `/api/kite/set-token`, `/auto-login`, and `/disconnect` strictly to SUPERADMIN. Remove `src/data/kite_session.json` from git tracking, rotate API keys, and eliminate TOTP console logging.
- **Correct Ledger Invariants on Trade Deletion (FIN-004):** Remove free-money margin refunds in `tradeController.deleteTrade`, ensuring that balance mutations occur exclusively through immutable, auditable ledger entries.

Phased Remediation Roadmap

PHASE	TARGET ITEMS	KEY ACTIONS & DELIVERABLES	ESTIMATED TIME
Phase 1: Urgent Security Perimeter	SEC-001, SEC-002, SEC-003, SEC-004, SEC-005, SEC-006, SEC-007, SEC-008, SEC-009, SEC-018	Delete root AI routes; add role checks on user management routes; parameterize SQL date filters; sanitize audio path traversal; restrict Kite admin routes; rotate secrets and remove TOTP logging.	1 - 2 Days
Phase 2: Money & Ledger Integrity	FIN-003, FIN-004, FIN-006, SEC-015, REL-005	Add hierarchy checks to internal transfers; eliminate bogus margin refunds in <code>deleteTrade</code> ; make transaction passwords mandatory; lock pending trade netting transitions.	2 - 3 Days

PHASE	TARGET ITEMS	KEY ACTIONS & DELIVERABLES	ESTIMATED TIME
Phase 3: Network & Session Lockdown	SEC-010, SEC-011, SEC-012, SEC-013, SEC-016, SEC-017	Enforce token verification on Socket.IO connect; add <code>express-rate-limit</code> to login and orders; implement refresh token rotation; protect <code>/uploads</code> static files; configure trust proxy hop counts.	2 - 3 Days
Phase 4: Reliability & Performance	PERF-001, PERF-002, REL-001, REL-002, REL-003, DASH-001, CQ-001, CQ-002	Add SQL pagination to <code>getUsers</code> ; decouple polling into tick stream listeners; add MySQL <code>GET_LOCK</code> to schedulers; centralize lot size lookup into single source of truth.	3 - 4 Days